What is a Function URL in AWS Lambda?

A **Function URL** is a dedicated HTTPS endpoint automatically generated by AWS for a Lambda function.

Instead of using API Gateway, AWS provides a direct URL that allows you to invoke your Lambda function through HTTP requests.

Example:

```
https://abc123xyz.lambda-url.ap-south-1.on.aws/
```

You can send:

- GET Requests
- POST Requests

directly to your Lambda function.

Architecture

```
Postman
   |
   | POST Request
   v
Lambda Function URL
   |
   v
AWS Lambda Function (Python)
   |
   v
Response
```

Task

Send two numbers from Postman:

```
{
    "num1": 20,
    "num2": 10
}
```

Lambda should return:

```
{
    "sum": 30,
    "difference": 10,
    "multiplication": 200,
    "division": 2
}
```

Step 1: Create Lambda Function

Choose:

```
Runtime: Python 3.13 (or latest available)
```

Step 2: Python Lambda Code

```
import json

def lambda_handler(event, context):

    body = json.loads(event["body"])

    num1 = body["num1"]
    num2 = body["num2"]

    result = {
        "sum": num1 + num2,
        "difference": num1 - num2,
        "multiplication": num1 * num2,
        "division": num1 / num2
    }

    return {
        "statusCode": 200,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps(result)
    }
```

How Parameters are Received

When Postman sends:

```
{
    "num1": 20,
    "num2": 10
}
```

AWS Lambda receives:

```
event = {
    "body": "{\"num1\":20,\"num2\":10}"
}
```

Notice that **body is received as a string**.

Therefore we convert it into a Python dictionary:

```
body = json.loads(event["body"])
```

Now we can access the values:

```
num1 = body["num1"]
num2 = body["num2"]
```

Understanding the Flow

Request from Postman

```
{
    "num1": 20,
    "num2": 10
}
```

Lambda Receives

```
event["body"]
```

Contains:

```
'{"num1":20,"num2":10}'
```

Convert String to Dictionary

```
body = json.loads(event["body"])
```

Result:

```
{
    "num1": 20,
    "num2": 10
}
```

Access Values

```
num1 = body["num1"]
num2 = body["num2"]
```

Perform Calculations

```
num1 + num2
num1 - num2
num1 * num2
num1 / num2
```

Send Response

```
return {
    "statusCode": 200,
    "body": json.dumps(result)
}
```

Step 3: Create Function URL

1. Open Lambda Function.
2. Click **Configuration**.
3. Select **Function URL**.
4. Click **Create Function URL**.

5. Authentication Type:

```
NONE
```

(Only for testing purposes)

AWS generates a URL such as:

```
https://abc123xyz.lambda-url.ap-south-1.on.aws/
```

Copy this URL.

Step 4: Configure Postman

Method

```
POST
```

URL

```
https://abc123xyz.lambda-url.ap-south-1.on.aws/
```

Headers

```
Content-Type : application/json
```

Body

Select:

```
raw
```

Choose:

```
JSON
```

Enter:

```
{
    "num1": 20,
    "num2": 10
}
```

Step 5: Send Request

Click **Send**.

Response:

```
{
    "sum": 30,
    "difference": 10,
```

```
    "multiplication": 200,
    "division": 2
}
```

AWS Lambda Cold Start and Warm Start Detection

This example helps identify whether the current request is a **Cold Start** or a **Warm (Hot) Start**.

How It Works

When AWS creates a new execution environment, the code outside the handler runs only once.

We use a global variable:

```
is_cold_start = True
```

- First request → Cold Start
- Subsequent requests using the same environment → Warm Start

Lambda Function Code

```
import json
from datetime import datetime

# Executed only once when the environment is created
is_cold_start = True

def lambda_handler(event, context):

    global is_cold_start

    if is_cold_start:
        start_type = "Cold Start"
        is_cold_start = False
    else:
        start_type = "Warm Start (Hot Start)"

    response = {
        "start_type": start_type,
        "request_time": str(datetime.now()),
        "request_id": context.aws_request_id
    }

    return {
        "statusCode": 200,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps(response)
    }
```

First Request

When the Lambda function is invoked for the first time:

Response

```
{
    "start_type": "Cold Start",
    "request_time": "2026-06-04 11:30:15",
    "request_id": "123456"
}
```

Why?

AWS had to:

1. Create an execution environment
2. Load Python runtime
3. Load your code
4. Execute the function

Second Request

Send another request immediately.

Response

```
{
    "start_type": "Warm Start (Hot Start)",
    "request_time": "2026-06-04 11:30:25",
    "request_id": "789101"
}
```

Why?

AWS reused the existing execution environment.

No initialization was required.

Third Request

```
{
    "start_type": "Warm Start (Hot Start)",
    "request_time": "2026-06-04 11:30:35",
    "request_id": "112233"
}
```

Again, AWS reused the same environment.

After Several Minutes

Suppose AWS removes the idle environment.

When a new request arrives:

```
{
    "start_type": "Cold Start",
    "request_time": "2026-06-04 11:45:20",
    "request_id": "445566"
}
```

A new execution environment was created.

AWS Lambda Environment Variables, Context Object, and S3 Event Trigger

1. What are Environment Variables in AWS Lambda?

Environment Variables are key-value pairs used to store configuration settings outside the source code.

Instead of hardcoding values such as database URLs, API keys, bucket names, or application settings inside the code, you can store them as environment variables and access them during runtime.

Advantages

- Improves security
- Makes code reusable
- Easy to change configuration without modifying code
- Different values can be used for Development, Testing, and Production environments

Example

Variable Name	Value
APP_NAME	Student Management System
VERSION	1.0
BUCKET_NAME	my-data-bucket

How to Create Environment Variables

1. Open AWS Lambda Console.
2. Select your Lambda Function.
3. Go to **Configuration**.
4. Click **Environment Variables**.
5. Click **Edit**.
6. Add Key and Value pairs.
7. Save changes.

Accessing Environment Variables in Python

Lambda Function Code

```
import os

def lambda_handler(event, context):

    app_name = os.environ.get("APP_NAME")
    version = os.environ.get("VERSION")

    return {
        "statusCode": 200,
        "body": {
            "Application": app_name,
            "Version": version
        }
    }
```

Example Output

```
{
  "statusCode": 200,
  "body": {
    "Application": "Student Management System",
    "Version": "1.0"
  }
}
```

2. What is Context Object in AWS Lambda?

When a Lambda function executes, AWS automatically provides two parameters:

```
def lambda_handler(event, context):
```

- **event** → Contains incoming request data.
- **context** → Contains runtime information about the Lambda execution.

Common Context Properties

Property	Description
context.function_name	Lambda function name
context.function_version	Function version
context.memory_limit_in_mb	Allocated memory
context.aws_request_id	Unique request ID
context.log_group_name	CloudWatch Log Group
context.log_stream_name	Log Stream Name
context.get_remaining_time_in_millis()	Remaining execution time

Context Example

```
def lambda_handler(event, context):

    return {
        "Function Name": context.function_name,
```

```
        "Function Version": context.function_version,
        "Memory": context.memory_limit_in_mb,
        "Request ID": context.aws_request_id,
        "Remaining Time": context.get_remaining_time_in_millis()
    }
```

Example Output

```
{
  "Function Name": "student-function",
  "Function Version": "$LATEST",
  "Memory": 128,
  "Request ID": "abc123xyz",
  "Remaining Time": 14998
}
```

Difference Between Event and Context

Event	Context
Contains incoming data	Contains Lambda runtime information
Sent by trigger	Sent automatically by AWS
Changes every request	Mostly execution metadata
Example: POST data, S3 upload data	Example: Function name, Request ID

3. How to Trigger AWS Lambda Using Amazon S3

AWS Lambda can automatically execute whenever an object is uploaded, deleted, or modified in an S3 bucket.

Architecture

```
User Uploads File
        |
        ▼
   Amazon S3
        |
        ▼
 Lambda Triggered
        |
        ▼
 Process File
        |
        ▼
 Store Result / Log
```

Steps to Configure S3 Trigger

Step 1: Create S3 Bucket

Example Bucket:

```
student-documents-bucket
```

Step 2: Create Lambda Function

Example:

```
def lambda_handler(event, context):

    print("S3 Event Received")

    return {
        "statusCode": 200
    }
```

Step 3: Add Trigger

1. Open Lambda Function.
2. Click **Add Trigger**.
3. Select **Amazon S3**.
4. Choose Bucket.
5. Event Type:

```
PUT
```

or

```
All Object Create Events
```

6. Save.

Understanding the S3 Event Object

When a file is uploaded, S3 sends an event object to Lambda.

Sample Event

```
{
  "Records": [
    {
      "eventName": "ObjectCreated:Put",
      "s3": {
        "bucket": {
          "name": "student-documents-bucket"
        },
        "object": {
          "key": "resume.pdf",
          "size": 1024
        }
      }
    }
  ]
}
```

Reading Bucket and File Information

```
def lambda_handler(event, context):

    bucket_name = event['Records'][0]['s3']['bucket']['name']
    file_name = event['Records'][0]['s3']['object']['key']

    print("Bucket:", bucket_name)
    print("File:", file_name)

    return {
        "statusCode": 200,
        "message": "File processed successfully"
    }
```

Example Output in CloudWatch Logs

```
Bucket: student-documents-bucket
File: resume.pdf
```

Complete Example: Environment Variables + Context + S3 Event

```
import os

def lambda_handler(event, context):

    app_name = os.environ.get("APP_NAME")

    bucket_name = event['Records'][0]['s3']['bucket']['name']
    file_name = event['Records'][0]['s3']['object']['key']

    print("Application:", app_name)
    print("Bucket:", bucket_name)
    print("File:", file_name)
    print("Function:", context.function_name)
    print("Request ID:", context.aws_request_id)

    return {
        "statusCode": 200,
        "message": "File processed successfully"
    }
```

Interview Questions

Q1. What are Environment Variables in AWS Lambda?

Environment Variables are key-value pairs used to store configuration settings separately from the code.

Q2. How do you access Environment Variables in Python Lambda?

```
import os

value = os.environ.get("VARIABLE_NAME")
```

Q3. What is the Context Object?

The Context Object contains runtime information about the Lambda function execution, such as function name, request ID, memory allocation, and remaining execution time.

Q4. What is an S3 Trigger?

An S3 Trigger automatically invokes a Lambda function when a file is uploaded, deleted, or modified in an S3 bucket.

Q5. Which parameter contains S3 event data?

`event`

Q6. Which parameter contains Lambda execution metadata?

`context`

Lambda Function Signature

```
def lambda_handler(event, context):
```

- **event** → Incoming data from trigger (S3, API Gateway, Function URL, etc.)
- **context** → Runtime metadata provided by AWS Lambda
- **Environment Variables** → Configuration values stored outside the code and accessed using `os.environ`

AWS Load Balancer with AWS Lambda

What is a Load Balancer?

A Load Balancer is a service that distributes incoming traffic across multiple targets to improve availability, scalability, and reliability.

In AWS, the most commonly used load balancer for Lambda is the **Application Load Balancer (ALB)**.

Traditionally, a load balancer sends traffic to:

- EC2 Instances
- Containers (ECS/EKS)
- IP Addresses

However, AWS also allows an **AWS Lambda Function** to be registered as a target of an Application Load Balancer.

Why Use a Load Balancer with Lambda?

Using an Application Load Balancer with Lambda provides:

1. HTTP/HTTPS Routing

Route requests based on:

- URL Path
- Host Name
- HTTP Headers
- Query Parameters

2. SSL/TLS Termination

The load balancer can handle HTTPS certificates so Lambda only receives the request.

3. Centralized Entry Point

Multiple Lambda functions can be accessed through a single endpoint.

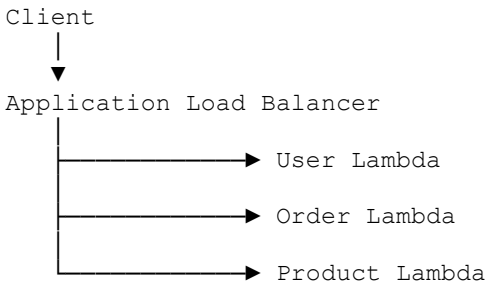
4. Microservices Architecture

Different paths can invoke different Lambda functions.

Example:

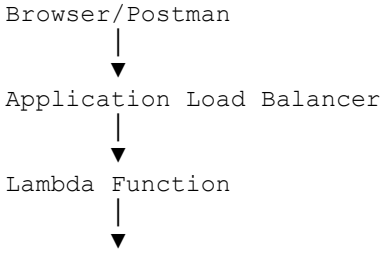
```
https://example.com/users      → User Lambda
https://example.com/orders     → Order Lambda
https://example.com/products   → Product Lambda
```

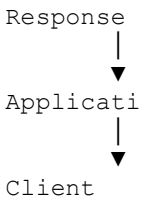
Architecture



How Lambda Works with ALB

Request Flow





Step 1: Create Lambda Function

Example:

```
import json

def lambda_handler(event, context):

    return {
        "statusCode": 200,
        "statusDescription": "200 OK",
        "isBase64Encoded": False,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps({
            "message": "Hello from Lambda through ALB"
        })
    }
```

Step 2: Create Target Group

1. Open AWS Console.
2. Go to EC2 Service.
3. Select **Target Groups**.
4. Click **Create Target Group**.

Choose:

```
Target Type = Lambda Function
```

Example:

```
Target Group Name = lambda-target-group
```

Click Next.

Step 3: Register Lambda Function

Select your Lambda Function.

Example:

```
student-api-function
```

Register it as a target.

Step 4: Create Application Load Balancer

Go to:

```
EC2 → Load Balancers → Create Load Balancer
```

Choose:

```
Application Load Balancer
```

Configure:

```
Name = student-alb
Scheme = Internet-facing
IP Type = IPv4
```

Select:

```
VPC
Subnets
Security Group
```

Step 5: Configure Listener

Create Listener:

```
HTTP : 80
```

or

```
HTTPS : 443
```

Default Action:

```
Forward To → lambda-target-group
```

Step 6: Test the Application

After creation AWS provides a DNS name:

```
http://student-alb-123456.ap-south-1.elb.amazonaws.com
```

Access it through browser or Postman.

Response:

```
{
  "message": "Hello from Lambda through ALB"
}
```

ALB Event Structure Received by Lambda

When a request reaches Lambda through ALB, AWS sends an event object.

Example:

```
{
  "requestContext": {
    "elb": {
      "targetGroupArn": "arn:aws:elasticloadbalancing:..."
    }
  },
  "httpMethod": "GET",
  "path": "/users",
  "queryStringParameters": {
    "id": "101"
  },
  "headers": {
    "host": "example.com"
  }
}
```

Reading Request Data

```
import json

def lambda_handler(event, context):

    path = event['path']
    method = event['httpMethod']

    return {
        "statusCode": 200,
        "statusDescription": "200 OK",
        "isBase64Encoded": False,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps({
            "path": path,
            "method": method
        })
    }
```

Path-Based Routing Example

Suppose you have three Lambda functions.

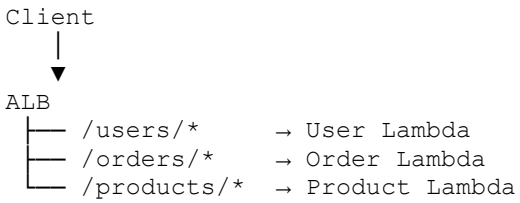
Lambda Functions

```
User Lambda
Order Lambda
Product Lambda
```

ALB Rules

```
/users/*     → User Lambda
/orders/*    → Order Lambda
/products/*  → Product Lambda
```

Architecture:



ALB vs API Gateway for Lambda

Feature	ALB	API Gateway
HTTP Routing	Yes	Yes
HTTPS Support	Yes	Yes
WebSocket Support	No	Yes
Request Validation	Limited	Advanced
API Management	Limited	Advanced
Cost	Usually Lower	Usually Higher
REST APIs	Basic	Excellent
Microservices Routing	Excellent	Excellent

AWS Lambda Function Name, File Name, and External Modules (Requests Library)

1. How to Change a Lambda Function Name

A Lambda function name is the unique name assigned to the function.

Method 1: Rename Using AWS Console

AWS Lambda does **not provide a direct Rename option**.

To change the function name:

1. Open AWS Lambda Console.
2. Open the existing function.
3. Download or copy the source code.
4. Create a new Lambda function with the desired name.
5. Paste or upload the code.
6. Copy environment variables, triggers, and permissions if needed.
7. Delete the old function (optional).

Example:

```
Old Function Name:
student-api
```

```
New Function Name:
student-management-api
```

2. What is the Lambda File Name?

When writing Lambda code, the handler format is:

```
filename.function_name
```

Example:

```
lambda_function.lambda_handler
```

Where:

- `lambda_function` = Python file name
- `lambda_handler` = Function name inside the file

Example

File: lambda_function.py

```
def lambda_handler(event, context):
    return {
        "message": "Hello AWS Lambda"
    }
```

Handler:

```
lambda_function.lambda_handler
```

Changing the File Name

Suppose you rename:

```
lambda_function.py
```

to

```
student_app.py
```

Then update the handler setting.

New Handler:

```
student_app.lambda_handler
```

Example

student_app.py

```
def lambda_handler(event, context):
    return {
        "message": "File renamed successfully"
    }
```

Handler Configuration:

```
student_app.lambda_handler
```

Changing Both File Name and Function Name

File

```
student_app.py
```

Code

```
def process_request(event, context):
    return {
        "message": "Custom handler"
    }
```

Handler:

```
student_app.process_request
```

Format:

```
filename.function_name
```

3. Using External Modules in AWS Lambda

AWS Lambda includes some built-in Python libraries.

Examples:

```
import json
import os
import datetime
import math
```

These work without installation.

What if a Module is Not Available?

Example:

```
import requests
```

The Requests library is not always available in the deployment package unless included by you.

You must package and upload it with your Lambda function.

Method 1: Upload ZIP Package

Step 1: Create Project Folder

```
lambda_project/
```

Step 2: Install Requests

Open terminal:

```
mkdir lambda_project

cd lambda_project

pip install requests -t .
```

The `-t .` option installs packages into the current folder.

Folder Structure

```
lambda_project/
│
├── requests/
├── urllib3/
├── certifi/
├── charset_normalizer/
├── idna/
└── lambda_function.py
```

Step 3: Create Lambda Code

lambda_function.py

```
import requests

def lambda_handler(event, context):

    response = requests.get(
        "https://jsonplaceholder.typicode.com/posts/1"
    )

    return {
        "statusCode": 200,
        "body": response.json()
    }
```

Step 4: Create ZIP File

Select all files inside the project folder and zip them.

Example:

```
lambda_package.zip
```

Step 5: Upload to Lambda

1. Open Lambda Function.
2. Upload ZIP package.
3. Deploy.

Method 2: Using Lambda Layers

Layers allow multiple Lambda functions to share libraries.

Benefits of Layers

- Reusable across many functions
- Smaller deployment packages
- Easier updates

Create Layer Folder Structure

```
python/
```

Install package:

```
pip install requests -t python/
```

Structure

```
python/
│
├── requests/
├── urllib3/
├── certifi/
└── idna/
```

Create ZIP

```
requests-layer.zip
```

Create Layer

1. Lambda Console
2. Layers
3. Create Layer
4. Upload ZIP
5. Select Python Runtime
6. Create

Attach Layer to Lambda

1. Open Lambda Function.

2. Layers Section.
3. Add Layer.
4. Select your Requests Layer.
5. Save.

Now you can directly use:

```
import requests
```

without packaging it every time.

Example: Calling a Public API

```
import requests

def lambda_handler(event, context):

    response = requests.get(
        "https://jsonplaceholder.typicode.com/users/1"
    )

    user = response.json()

    return {
        "statusCode": 200,
        "body": {
            "name": user["name"],
            "email": user["email"]
        }
    }
```

Example Using Environment Variable

```
import os
import requests

def lambda_handler(event, context):

    api_url = os.environ.get("API_URL")

    response = requests.get(api_url)

    return {
        "statusCode": 200,
        "body": response.json()
    }
```

Environment Variable:

```
API_URL=https://jsonplaceholder.typicode.com/posts/1
```

Common Import Errors

Error

```
Unable to import module 'lambda_function'
```

Reason:

- Wrong handler name
- File not found
- Syntax error

Error

```
No module named 'requests'
```

Reason:

- Requests package not included
- Layer not attached

Solution:

```
pip install requests -t .
```

or use a Lambda Layer.

Quick Revision

Handler Format

```
filename.function_name
```

Example:

```
student_app.process_request
```

Rename File

```
lambda_function.py
```

↓

```
student_app.py
```

Update handler accordingly.

Install Requests

```
pip install requests -t .
```

Create Layer

```
pip install requests -t python/
```

Import Requests

```
import requests
```

Use Requests

```
response = requests.get("https://api.example.com")
data = response.json()
```

What are AWS Lambda Layers?

A Lambda Layer is a separate package that contains libraries, dependencies, custom runtimes, or shared code that can be used by multiple Lambda functions.

Instead of including the same libraries in every Lambda deployment package, you can place them in a Layer and attach that Layer to one or more Lambda functions.

Why Use Layers?

Imagine you have 10 Lambda functions and all of them use the Requests library.

Without Layers:

```
Lambda 1 → Requests Library
Lambda 2 → Requests Library
Lambda 3 → Requests Library
...
Lambda 10 → Requests Library
```

The same library is uploaded 10 times.

With Layers:

```
Requests Layer
      │
      ├── Lambda 1
      ├── Lambda 2
      ├── Lambda 3
      ├── Lambda 4
      └── Lambda 10
```

The library is uploaded only once and shared among all functions.

Benefits of Layers

- Reusable code
- Smaller deployment packages
- Easier maintenance
- Faster deployments
- Share libraries across multiple functions

What Can Be Stored in Layers?

Python Libraries

```
requests
numpy
pandas
openpyxl
```

Shared Utility Code

```
database.py
helper.py
logger.py
```

Custom Runtime

```
Node.js Runtime
Python Runtime
Java Runtime
```

Layer Architecture

```
Lambda Function
      │
      ▼
Attached Layers
      │
      ├── Requests Library
      ├── Pandas Library
      └── Utility Functions
```

When Lambda starts, AWS automatically loads the Layer contents.

Creating a Python Layer

Step 1: Create Folder

```
mkdir python
```

Step 2: Install Library

Example:

```
pip install requests -t python/
```

Folder Structure:

```
python/
│
├── requests/
├── urllib3/
```

```
├── certifi/
└── idna/
```

Step 3: Create ZIP

```
requests-layer.zip
```

Zip the `python` folder.

Step 4: Create Layer

AWS Console:

```
Lambda
  → Layers
      → Create Layer
```

Upload:

```
requests-layer.zip
```

Choose:

```
Compatible Runtime:
Python 3.x
```

Create Layer.

Step 5: Attach Layer

```
Lambda Function
    → Layers
        → Add Layer
```

Select:

```
requests-layer
```

Save.

Using the Layer

Now the library can be imported directly:

```
import requests

def lambda_handler(event, context):

    response = requests.get(
        "https://jsonplaceholder.typicode.com/posts/1"
    )

    return response.json()
```

No need to package Requests again.

Layer Versions

Each time you update a Layer, AWS creates a new version.

Example:

```
Requests Layer

Version 1
Version 2
Version 3
```

Older Lambda functions can continue using older versions if required.

2. What is Versioning in AWS Lambda?

Versioning creates immutable snapshots of your Lambda function.

A Version stores:

- Function Code
- Environment Variables
- Memory Settings
- Timeout Settings
- Layers Configuration

Once published, a Version cannot be modified.

Why Use Versions?

Suppose your function is working perfectly.

Current code:

```
def lambda_handler(event, context):
    return "Version 1"
```

Publish:

```
Version 1
```

Later you update:

```
def lambda_handler(event, context):
    return "Version 2"
```

Publish again:

```
Version 2
```

Now both versions exist independently.

Lambda Version Architecture

```
$LATEST
    │
    ├── Version 1
    ├── Version 2
    └── Version 3
```

What is $LATEST?

AWS always keeps a working copy called:

```
$LATEST
```

Whenever you edit code:

```
Changes → $LATEST
```

Until you publish a version.

Publishing a Version

AWS Console:

```
Lambda Function
    → Actions
        → Publish New Version
```

Example:

```
Version: 1
Description:
Initial Release
```

Example

```
$LATEST → Under Development

Version 1 → Stable Release

Version 2 → Bug Fix

Version 3 → New Features
```

Benefits of Versioning

- Rollback capability
- Safe deployments
- Release management
- Testing multiple releases

3. What is Alias in AWS Lambda?

An Alias is a friendly name that points to a specific Lambda Version.

Instead of using version numbers directly, applications can use aliases.

Problem Without Alias

Direct Invocation:

```
Function: student-api:1
```

After update:

```
Function: student-api:2
```

Every client must change references.

Solution Using Alias

Create:

```
PROD
```

Alias pointing to:

```
Version 1
```

Clients call:

```
student-api:PROD
```

Later:

```
PROD → Version 2
```

Clients continue using:

```
student-api:PROD
```

No code changes needed.

Alias Architecture

```
Versions

Version 1
Version 2
Version 3

Aliases
```

```
DEV  → Version 1
TEST → Version 2
PROD → Version 3
```

Common Alias Names

```
DEV
TEST
QA
STAGING
PROD
```

Creating an Alias

AWS Console:

```
Lambda Function
    → Aliases
        → Create Alias
```

Example:

```
Alias Name: PROD

Version:
3
```

Traffic Shifting with Alias

One powerful feature of aliases is traffic splitting.

Example:

```
PROD Alias

90% → Version 1
10% → Version 2
```

Architecture:

```
Client
   │
   ▼
PROD Alias
   │
   ├── 90% → Version 1
   └── 10% → Version 2
```

Useful for:

- Blue-Green Deployment
- Canary Deployment
- A/B Testing

Relationship Between Layers, Versions, and Aliases

```
Lambda Function
       |
       ▼
Layers Attached
       |
       ▼
Publish Version
       |
       ▼
Create Alias
       |
       ▼
Production Access
```

Real-World Example

Suppose you have:

```
Function:
student-api
```

Attached Layer:

```
requests-layer
```

Publish:

```
Version 1
```

Create Alias:

```
PROD → Version 1
```

After adding new features:

```
Update Code
```

Publish:

```
Version 2
```

Move Alias:

```
PROD → Version 2
```